IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received March 27, 2019, accepted April 14, 2019, date of publication April 29, 2019, date of current version May 20, 2019.

Digital Object Identifier 10.1109/ACCESS.2019.2913902

Congestion Control and Packet Scheduling for Multipath Real Time Video Streaming

SONGYANG ZHANG, WEIMIN LEI, WEI ZHANG, YUNCHONG GUAN, AND HAO LI
Department of Communication and Electronic Engineering, School of Computer Science and Engineering, Northeastern University, Shenyang 110169, China
Corresponding author: Weimin Lei (leiweimin@ise.neu.edu.cn)

This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB1702000, in part by the Liaoning Provincial Natural Science Foundation of China under Grant 20180551007, and in part by the National Natural Science Foundation of China under Grant 61671141.

ABSTRACT Real time video traffic has stringent delay requirement and is bandwidth demanding when streaming over the internet. Single path transmission may provide unsatisfactory picture quality to users due to packet loss, jitter, and low bandwidth. Multipath transmission is another option to achieve better service by aggregating bandwidth and exploiting path diversity. As utility maximization theory shows, to achieve better satisfaction is to provide higher throughput and lower frame delivery delay for real time video traffic. The congestion control is responsible to probe maximum available bandwidth and packet scheduling is to take advantages of path diversity, which are two key research points in multipath context and are also the main concern in this paper. Given the BBR congestion control algorithm causes high-packet loss rate and large transmission delay when BBR flows competing for bandwidth resource, a delay response BBR (Delay-BBR) algorithm is proposed for real time video transmission. The main idea is that the sender will actively reduce the rate to let intermediate routers drain the excess occupied buffer once round trip delay signal exceeds a certain threshold. The results show that the delay-BBR can achieve lower transmission delay and lower packet loss rate when compared with QUIC-BBR and WebRTC-BBR. The rate stable feature makes delay-BBR appropriate for real time video transmission. Moreover, a packet scheduling algorithm works in coordination with the rate control algorithm achieves lower frame delivery delay further compared with the benchmark algorithms.

INDEX TERMS Multipath transmission, load distribution, real time video traffic, congestion control.

I. INTRODUCTION

According to a recent report [1], video traffic will grow up more than 80% of all internet traffic by 2022. The Quality of Experience (QoE) of video streaming is still a main concern for applications developers and service providers. Video transmission in current network still suffers from restricted bandwidth, packet loss. Low quality image could be delivered due to bandwidth limitation. Packet loss has different consequences such as re-buffering, throughput decrease, or mosaic according to whether video traffic is streaming over reliable or unreliable mode. Such unfavorable factors are quite harmful to QoE.

There are two widely applied technologies DASH (dynamic adaptive streaming over HTTP) and WebRTC[1] to optimize video transmission. DASH is mainly adopted by video content providers e.g., YouTube and NetFlix, while WebRTC is used to build interactive multimedia applications such as conference, video calls. Both of them could dynamically adjust video bitrate according to current network status. In DASH system, a duration of video content is encoded into different bitrate chunks. A client requests appropriate chunks based on estimated bandwidth and the length of buffered video to avoid rebuffering. In WebRTC, the bitrate and resolution of an encoded video frame is dynamically changed as the requirement of congestion control algorithm.

There exists another possibility to provide better QoE by exploiting multipath transmission. Today's networks are becoming multipath. Mobile devices equipped with multi-homed interfaces make concurrent access to heterogeneous networks (WIFI and 4G) possible. Datacenter networks provide redundant paths for high availability concern. Enterprises would choose a secondary internet service provider to reduce the risk of internet outage. Multipath transmission scheme brings out many advantages such as

The associate editor coordinating the review of this manuscript and approving it for publication was Shagufta Henna.

[1]https://webrtc.org/

2169-3536 © 2019 IEEE. Translations and content mining are permitted for academic research only.
Personal use is also permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

providing aggregating bandwidth, accelerating flow completion time and improving robustness. There are two transport layer protocols MPTCP and CMT-SCTP to maximize resource usage in multipath context. The two protocols transmit packets in reliable mode and are suitable for bulk data transmission. For real time video streaming, UDP is a more preferable choice and video frames can be delivered in unreliable mode for latency consideration. Skipping some of incomplete none key frames may not bring perceptible quality deterioration due to packet loss but latency is quite annoying for real time video applications. Real Time Protocol (RTP) is extended in [2] to enable multipath transmission for real time video streaming.

Two key research points on multipath transmission scheme are congestion control and packet scheduling, which are also the main consideration in this article. There are several fruitful results on congestion control for MPTCP such as LIA [3], OLIA [4], wVegas [5]. The scheduling algorithms [6]–[10] for MPTCP aim to improve throughput and minimize flow completion time by taking the idea to deliver packets out of order to achieve in order arrival. Due to heterogeneity of routing paths, packets sent through different paths may arrive out of order at receiver, which would occupy buffer resource and may cause head of line blocking (HOL) [11]. Once the pre-allocated buffer is fully occupied, the subsequent packets will be dropped. HOL blocking leads the throughput of MPTCP suboptimal since the rate backoff after packet loss detection is not caused by network congestion. Since buffer can be allocated flexibly at application layer, packet scheduling for multipath real time video transmission is not suffering from HOL, and delay bound should be considered in priority.

The congestion control algorithms for MPTCP are variant of Additive Increase Multiplicative Decrease (AIMD) law [12] adapted for multipath context. The congestion window reduced by half due to packet loss makes packet sending rate show saw-tooth feature. It would result instability of video encoder if applied in real time video transmission. GCC [13] implemented in WebRTC prefers a rate back off factor 0.8 when link is in congestion to maintain stable video frame generating bitrate.

Inspired by the excellence performance of a most recently proposed congestion control algorithm BBR [14], a delay response BBR (Delay-BBR) optimized for real time video transmission is proposed to overcome the drawbacks of high packet loss rate and longer transmission latency. By actively reducing sending rate to let intermediate routers drain the occupied buffer when the round trip delay signal exceeds a pre-defined threshold, Delay-BBR can achieve obvious lower packet loss rate and lower transmission latency while maintaining reasonable throughput at the same time. Moreover, a packet scheduling algorithm deduced from utility maximization theory is proposed, which takes the local occupied buffer length, available bandwidth and transmission delay into consideration. In our implementation, the congestion control algorithm and packet scheduling algorithm work on application layer. The real time video traffic is transmitted via UDP.

Simulation results show the congestion control algorithm can converge fast to available bandwidth, maintain rate stability and guarantee fairness when sharing bottleneck with other flows exploiting the same congestion control algorithm. The packet scheduling algorithm achieves the lowest average frame delivery delay when compared with other benchmark algorithms.

The rest of this paper is organized as follows. Section 2 provides an overview of related works on congestion control algorithms and packet scheduling algorithms. The propose multipath transmission framework for real time video is introduced in Section 3. The implementation detail on congestion control algorithm and packet scheduling algorithm are described in Section 4. The performances evaluation and comparison are presented on Section 5. Section 6 is the conclusion and future working directions.

II. RELATED WORK

As pointed by [15], all flows across internet should implement rate control scheme for congestion avoidance and promote fair bandwidth occupation. Without restriction of congestion control algorithms, network users would send packets with rate as highly as possible in egoistic manner and network would fall risk into collapse.

Jocobson recommended to regulate sending rate of TCP flows according to additive increase and multiplicative decrease (AIMD) law in [12], which later developed into TCP Reno algorithm. On every RTT, one more packet could be sent into network to probe more bandwidth. Reno takes packet loss as network congestion signal and multiplicatively reduces the size of congestion window by half when packet loss happens to alleviate link congestion. Most of the later algorithms such as Bic [16], Cubic [17], revise the bandwidth probe mechanism to make the control algorithm converge quickly to maximum available bandwidth or attempt to solve RTT unfairness issue.

The main goals of congestion control for MPTCP are to guarantee intra-protocol friendliness when the subflows of a multipath session share a common bottleneck and move as much as traffic off the congested paths [3]. These multipath congestion control algorithms such as LIA, OLIA, wVegas are built on basis of AIMD rate control mechanism. They couple all subflows together to maintain bottleneck fairness when more than one subflows of a multipath session traversing a common bottleneck and competing bandwidth with single path TCP session.

These congestion control algorithms applied in TCP or MPTCP taking packet loss to infer link congestion is not quite fit for real time video transmission. First, the saw-tooth feature of sending rate of loss based congestion control algorithms due to half reduction of congestion window would lead instability of video encoder, which would generate video pictures with fluctuation quality. Second, in cellular access network, there exists random packet loss caused by

interference or channel fading, and the throughput of loss based rate control algorithms is low in this situation. Third, in wire network, packet loss is a rare event since deep buffer is configured in modern routing devices. These loss based congestion control algorithms tend to occupy much buffer resource in routers once the rates of all users exceed the capacity of bottleneck and result in longer transmission delay, which is notorious for bufferbloat [18].

In consideration of these drawbacks of AIMD congestion control algorithms, there have been a tendency to deploy congestion control algorithms at application layer for real time video transmission. IETF has initiated RMCAT (RTP Media Congestion Avoidance Techniques) Working Group to develop congestion control mechanisms for interactive real-time media. There are three congestion control drafts under this working group, namely, GCC [13], NADA [19], SCReAM [20]. All three algorithms take delay related signal to infer congestion, which can response link congestion earlier than loss based algorithms. GCC takes one way delay gradient as congestion signal, NADA takes an aggregated delay signal and SCReAM takes one way delay signal to control rate. These algorithms pursuit to maintain reasonable throughput when sharing links with other type flows, while keeping a low link queue occupancy at the same time.

The recent research works on packet scheduling algorithm for multipath transmission are briefed in following.

DEMS [7] is a chunk based scheduling algorithm for MPTCP to achieve simultaneous sub-flow completing time. The length of a chunk is decided by application layer according to estimated bandwidth and one way delay. STMS [9] schedules packets with small sequence number to fast path while sending packets with large sequence number on slow path, to solve packets out of order arrival problem. The main idea of STMS is quite similar to DEMS with difference that STMP is implemented on transport layer. In [10], the authors designed a cross layer packet scheduling algorithm which takes packet queue length and packet pending delay in network interface card (NIC) into consideration. When a packet comes, the scheduler would choose the NIC that can deliver the packet earliest to destination.

The authors of OMS [22] proposed to schedule packets to path with minimal arrival delay. The packet arriving delay is calculated by link capacity and buffered queue length in routers. In real network situation, link capacity can be measured by packet pair like technology [23], but queue occupation length in links is hard to estimate. In their simulation, the queue occupation following asymptotically Weibull distribution [24] is assumed and routing paths have translational Brownian motion (tBm) traffics with prior known mean rate, variance coefficient and Hurst parameter. This algorithm is difficult to be practically applied in real network since these parameters are hard to estimate from an endpoint.

In [25], packet scheduling combined with rate control scheme is proposed for mobile devices which can access heterogeneous wireless networks concurrently. A rate distortion equation is introduced as the optimization goal, which is related to available bandwidth and packet loss rate. The packet loss rate is modeled from M/M/1 queue theory. Whether the M/M/1 model can reflect the true status of network link is quite questionable. The idea to minimize rate distortion is further developed in Wu' work [26] and packets are distributed to different paths on frame level. A TCP friendly congestion control algorithm is applied in Wu's work.

In [27], Markov Decision Process is applied for packet distribution over multipath overlay network. The incoming packets are segmented into fixed size bins before delivered to traffic distributor to choose path. The objective of MDP approach is to maximize the average reward over a short time span. The immediate reward when packets are sent by a specific path is defined as the average packet inter-departure time and propagation delay.

EDCLD [28] takes a hybrid model to compute the path cost, which is modeled from Poisson traffic and unknown traffic. Its goal is to minimize cost variation from different paths though packet scheduling. Increase packet scheduling ratio on path with smaller cost and decrease it on path with larger cost. Due to constraint of available bandwidth, scheduling more packets to path with smaller cost would increase packets transmission delay. Finally the costs are balanced on all paths.

SFL [29] splits video frames into sub-frames to optimize delivery delay in wireless network. Watering filling algorithm is applied, and packets are scheduled to specific path according to available bandwidth and one way delay to make equal aggregate delay in all paths.

Nearly all of these algorithms share a common idea that the optimization is achieved by scheduling more packets to path with less cost and the core differences of these works are the definition of routing path cost which may be expressed by delay, packet loss rate, or both. And we admit here, the main idea of proposed packet scheduling algorithm also belongs to such kinds. But we argue here, due to the different definition of cost function, the performances of packet scheduling algorithms show diversity in simulation.

III. FRAMEWORK

A. THE MULTIPATH TRANSMISSION SYSTEM FOR REAL TIME VIDEO

The multipath real time video transmission framework applied in this work is shown in Figure 1. The most important module is congestion controller (CC), which is introduced to guarantee fair share of bandwidth when real time video flow competing resource with other types flow and to avoid leading network into congestion. The congestion controller is implanted on each sub-path and only one is showed in diagram due to limited space.

The pacer sends these packets in local buffer into network evenly with rate determined by congestion controller. Traditional window based congestion control algorithms send packets in burst mode, which may lead these sent packets queued at intermediate routers and introduce extra delay.

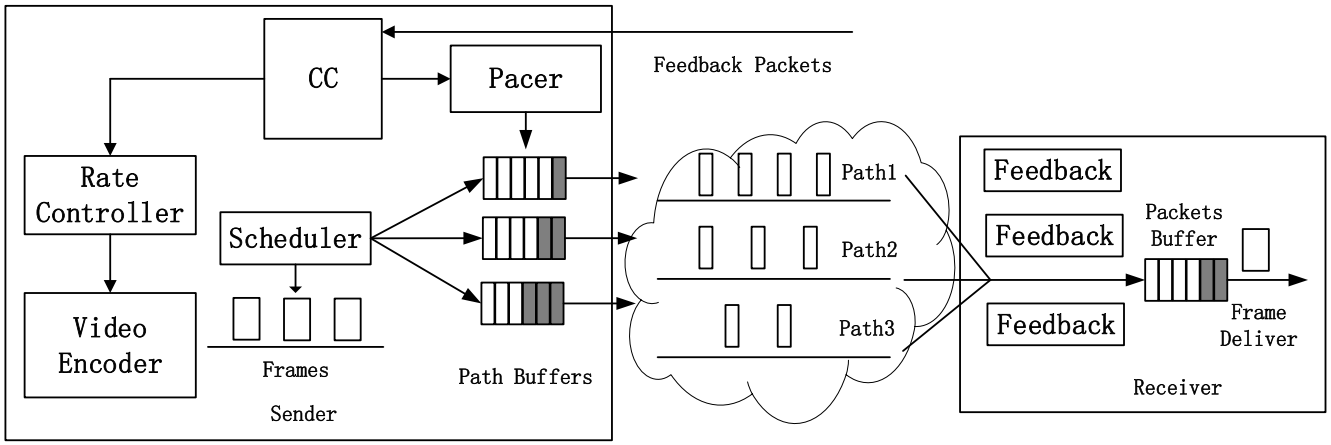


FIGURE 1. The designed multipath transmission framework.

There is a tendency to implement a pacing module in newly proposed algorithms e.g., BBR and GCC. There are works [30], [31] argue about the benefit of pacing.

The scheduler is responsible for sending packets to local buffer of different paths according to some rules. The rate controller adjusts the bitrate of the video encoder to make sure the output rate of encoded video frame matches the total available throughput of all its sub-paths.

At receiver side, the feedback module on each sub-path would send acknowledgement packets to sender. They are fed to congestion controller to infer whether routing path is in congestion or not and packet sending rate is adjusted accordingly. Packets sent on different path belonging to a same frame would arrive out of order at destination in large possibility. A jitter buffer is configured at receiver to regroup packets and video frames can be delivered to upper layer in evenly spaced intervals.

Real time video traffic is usually transported over UDP for latency consideration and our experiments are no exception. The multipath video transmission protocol in simulation is briefed, which is taken reference from razor project.[2] The incoming video frame is packaged into segments. Apart from video data, some extra information is included in the segment header: The frame id (fid), the total packaged segments of a frame, the segment index in a frame, the video frame generating timestamp. The segment index is used for putting the received segment in right position in receiver buffer.

B. PROBLEM FORMULATION

The utilization function U is introduced to find methods to provide better satisfaction of a service, which is usually assumed as a strictly concave and increasing function. The network system is modeled as a set of L routing paths. S denotes the number of network users. A user s can exploit multiple routing paths to build concurrent connections. $x_{s,l}$ is the packet sending rate of user s on path l, which is zero if l is not used by s. The total rate of user s is $y_s = \sum_{l \in L} x_{s,l}$. The bottleneck capacity (the narrowest link) of path l is c_l. User s will get a satisfaction or welfare measured by $U(y_s)$ when transmitting packet with rate y_s. The goal is to find a reasonable bandwidth allocation scheme to get resource fully utilized. The following optimization problem is obtained:

$$\begin{aligned} &\max \sum_{s \in S} U(y_s) \\ &\text{s.t.} \sum_{s \in S} x_{s,l} \le c_l \end{aligned} \tag{1}$$

A global bandwidth allocation solution is quite hard considering the scale and heterogeneity of current network systems. Such problem is usually solved by deploying rate control algorithms at endpoints. The Lagrangian is defined as following:

$$\begin{aligned} L(y_s, \lambda) &= \sum_{s \in S} U(y_s) + \sum_{l} \lambda_l (c_l - \sum_{s \in S} x_{s,l}) \\ &= \sum_{s \in S} U(y_s) - \sum_{l} \lambda_l (\sum_{s \in S} x_{s,l}) + \sum_{l} \lambda_l c_l \\ &= \sum_{s \in S} U(y_s) - \sum_{s \in S} \sum_{l} x_{s,l} \lambda_l + \sum_{l} \lambda_l c_l \end{aligned} \tag{2}$$

The parameter λ_l is the shadow price introduced by Kelly [32] of path l. More specifically, the shadow price is congestion signal feedback by network system, which may be increased delay or packet loss event due to channel overuse. According to previous research works [32] [33] [5], the dual function of the original problem is:

$$\min_{\lambda} D(\lambda) \tag{3}$$

The function $D(\lambda)$ is defined as:

$$D(\lambda) = \max L(y_s, \lambda) = \sum_{s \in S} L_s(\lambda) + \sum_{l} \lambda_l c_l \tag{4}$$

where

$$L_s(\lambda) = \max U(y_s) - \sum_{l} x_{s,l} \lambda_l \tag{5}$$

[2]https://github.com/yuanrongxi/razor

Equation (5) is defined as aggregate surplus in [34]. To achieve the maximization of aggregate surplus is to send packets with rate as highly as possible while maintaining minimal link price at the same time. Due to capacity limitation of bottleneck, users send packets with too high rate may lead network congestion, which would increase the cost in the second part of the equation (5) and lead aggregate surplus decrease. Usually, a congestion control algorithm increases rate to probe more bandwidth and responses link congestion by rate reduction, through which the stability of a network system can be maintained. In multipath context, to maximize aggregate surplus is to minimize packet transmission cost with constraint of congestion control algorithm . As for the congestion control algorithm, an improved BBR optimized for real time video transmission is implemented in this work. To minimize transmission cost is the responsibility of the packet scheduling module, which is the main focus in following analysis.

To make the utilization theory can be well applied in multipath packet scheduling problem, a minor modification is made to equation (5).

$$\max\ U(x) - \sum_{p} \lambda_p x_p \tag{6}$$

where, p is path index, λ_p is path price, and x_p is the packet scheduling rate. The packets generating rate is x, $x_p = \alpha_p x$. α_p is rate splitting ratio.

The aggregate cost of multipath session is denoted by q, which can be interpreted as the expected cost of a single packet scheduled through multipath (8).

$$xq = \sum_{p} x_p \lambda_p \tag{7}$$

$$q = \sum_{p} \lambda_p \alpha_p \tag{8}$$

The path price λ_p can be denoted by (9). d_p is fixed propagation delay. $d_q(t)$ is packets queueing delay at intermediate routers. And the third term is delay introduced by packets queued in local buffer at sender side, $\mathbf{1}(\cdot)$ is the step function. Once the packet scheduling rate $x_p(t)$ exceeds the path sending rate c_p, extra sent packets would be queued at local buffer.

$$\lambda_p(t) = d_p(t) + d_q(t) + \int \frac{(x_p(t) - c_p(t))\mathbf{1}(x_p(t) - c_p(t))}{c_p(t)} \mathrm{d}t \tag{9}$$

As for the first term of equation (6), there is quite little a packet schedule algorithm can wield impact. The main focus of packet scheduling algorithm is to minimize the second term.

$$q_t = \boldsymbol{\lambda_{p,t}} \boldsymbol{\alpha_{p,t}} \tag{10}$$

The aggregate cost equation (8) can be rewritten as equation (10). $\boldsymbol{\alpha_{p,t}}$ works as an indicator to schedule an incoming packet to a specific path. It's obvious that to minimize the aggregate cost is to schedule a packet to path with minimal path price in current time. Assuming the frame generating rate is set as the sum of available bandwidth of all the sub-paths $x = \sum_p c_p$, it means a single path cannot always sending the incoming packets to its peer within video frame rendering deadline. If more packets are scheduled to the minimal cost path, packets would be buffered once these packets can not be sent out immediately and the path price would further increase as equation (9) shows. There is a time point that packets would be switched to another path. Such method can finally strike a balance in terms of path price in all paths.

IV. IMPLEMENTATION

The implementation on congestion control algorithm will be detailed in this part. To evaluate the performance of the proposed packet scheduling solution, several benchmark algorithms are implemented. The multipath real time video transmission framework shown in Figure 1 is implemented on ns3.[3]

A. IMPLEMENTATION ON CONGESTION CONTROL ALGORITHM

In an early stage when we search a congestion control solution for multipath real time video transmission, GCC algorithm is applied. We observed in simulation that the rate in underload path ramps up slowly during initial phase and the total throughput is sub-optimal. The packet scheduling algorithm delivers packets to path p_a with smaller price and the incoming packets will be switched to another path p_b until the price of p_a is larger than p_b due to buffer occupation. Before switch happens, there are not enough packets to be sent on p_b. It affects bandwidth estimation in GCC and results in low throughput on path with larger transmission delay.

In such dilemma situation where GCC can not work well in multipath context, the recent proposed congestion control algorithm BBR is tested, which is said to achieve remarkable performance for bulk data transfer in TCP. Basically, BBR is claimed as a congestion based rate control algorithm, with goal to reach the optimal control point, namely, to achieve the maximum throughput while minimizing end to end latency at the same time. The sending rate of BBR flow is depended on estimated bandwidth gained from acknowledgement packets. It does not response to link delay increase nor packet loss event. But simulation results indicate the packets loss rate is high and round trip time is large when several BBR flows competing for bandwidth in the same routing path. The reason is analyzed in fourth paragraph in this part.

There are four control states StartUp, Drain, ProbeBW, and ProbeRTT in BBR. The Startup and Drain states are used at session start. At the Startup phase, the endpoint uses a high gain $\frac{2}{ln2}$ to double its sending rate to probe the maximum available bandwidth. When estimated bandwidth is 1.25 times less than the previous sending rate and lasts for 3 times, the sender deems it approach to the maximum available bandwidth and enters into Drain state. The Drain

[3]https://www.nsnam.org/

state is to get rid of excess queue accumulated during StartUp phase. When the inflight packets are less than BDP (Bandwidth Delay Product), the state is changed to ProbeBW. At this state, the sending rate is controlled by different gains [1.25, 0.75, 1, 1, 1, 1, 1, 1] in 8 RTTs. The gain 1.25 is to probe bandwidth. If there is extra bandwidth available, the sender would increases sending rate to occupy it. If the minimal RTT is not sampled again within 10 seconds, the link seems in congestion and the state is set as ProbeRTT. Only four packets are allowed to be sent out in a RTT to let intermediate routers drain the occupied buffer and the minimal RTT is reset.

Some features of BBR algorithm are unsuitable to be applied directly in real time video transmission. First, the endpoint would send too many packets with pacing gain 1.25 in ProbeBW. At the next RTT, the transmission rate would be reduced to 0.75 times less than its throughput to avoid excess queue. Due to video frames are captured at fixed intervals and the output bitrate of video encoder is stable in a short time span, there may be not enough packets to be sent to probe bandwidth in 1.25X gain phase while the packets may be queued in the local buffer in 0.75X gain phase. Second, all BBR flows would insist the maximum estimated bandwidth sampled during last 10 RTTs and the total sending rates tend to exceed the capacity of bottleneck. The extra sent packets would be buffered in routers and lead transmission latency increase. The minimal RTT could hardly appear again in 10s. The ProbeRTT state which lasts at most 200 milliseconds may be not longer enough to let routers empty the occupied buffer. Thus the optimal control point is deviated. Third, only four packets are sent in a RTT during ProbeRTT would further introduce frame transmission delay in video transmission.

Given these drawbacks, a delay response BBR (Delay-BBR) optimized for real time video transmission is proposed. First, the pacing gains [1.25, 0.75, 1, 1, 1, 1, 1, 1] are changed to [1.11, 0.9, 1, 1, 1, 1, 1, 1] to maintain stable sending rate. The 1.25X pacing gain shows quite aggressiveness when flows competing for bandwidth. Second, as for optimal control point deviation problem due to excess queue occupation, instead of only entering the ProbeRTT state when the minimal RTT is not sampled in 10s, we found that let the sender enter the ProbeRTT state more frequently can gain better performance.

In Delay-BBR algorithm, once the smoothed delay signal exceeds a certain threshold, the sending rate will be reduced to let routers empty occupied buffer to get network system recover back to optimal control point. The smoothed RTT (srtt) signal is monitored in ProbeBW state as Equation (11). α is 0.9 and the exponential filter is applied to eliminate noise. Once srtt is β (1.2 in implementation) times larger than the base RTT (base_line_rtt_, the minimal RTT in ProbeBW), the link seems fall into congestion (line 4 in Algorithm 4), the sending rate would be reduced to $0.75X$ (line 6 in Algorithm 5) of its maximum rate before. Until inflight packets are less than BDP (bw*min_rtt_), the state would be changed into ProbeBW (line 11 in Algorithm 5). This gives routers enough time to drain the occupied buffer. In our implementation, the purpose of ProbeRTT state is not exploited to sample a new minimal RTT but for the excess buffered packets to be drained. The state may be changed to ProbeRTT for many times during the minimal RTT monitoring interval in the modified version. Once the control state is set as ProbeRTT, srtt would be reset as zero (line 4 in Algorithm 5) temporarily and the sequence number of the last sent packet is recorded as seq_at_backoff_ (line 3 in Algorithm 5). The srtt signal will be updated until the acknowledged sequence number is larger than the seq_at_backoff (line 11-16 in Algorithm 3). Such operation is to maintain reasonable rate when real time video flows sharing links with loss based congestion control constraint flows.

$$srtt = (1 - \alpha) * srtt + \alpha * rtt \tag{11}$$

$$bw = min(\frac{\Delta sent}{\Delta sent_ts}, \frac{\Delta acked}{\Delta ack_ts}) \tag{12}$$

When a packet is sent out by sender, the packet length and sent timestamp will be recorded in Algorithm 1. Here, inflight_ is the total length of sent packets without being acknowledged. On each acknowledgement, the sender would compute bandwidth as Equation (12), update rtt following Algorithm 2, change packets inflight counter in Algorithm 3, check whether the link is in congestion status in Algorithm 4. When link is in congestion or the minimal rtt monitor interval expires (kMinRttExpiry is 10 seconds), the rate pacing gain will set as 0.75 in Algorithm 5 to let routers drain buffered packets in exchange for lower queueing latency.

Algorithm 1 OnPacketSent

Require:
the timestamp (now), sequence (seq) and the payload length of a sent packet

```
1: info.sent_ts=timestamp,info.bytes=payload
2: sent_packets_map_.insert(seq,info)
3: inflight_← inflight_+payload
4: last_sent_packet_←seq
```

Algorithm 2 OnAck

Require:
the ack packet received timestamp (now) and acknowledged sequence number (seq)

```
1: UpdateRttAndInflight(now,seq)
2: congested←CheckIfCongestion()
3: if now-min_rtt_ts_>kMinRttExpiry then
4:     min_rtt_expired← 1
5: end if
6: DrainExcessBuffer(now,min_rtt_expired,congested)
```

Even though I tried to explain the implementation detail in both word and pseudo code, limitation of language expressive

Algorithm 3 UpdateRttAndInflight

Require:
the ack packet received timestamp (now) and acknowledged sequence number (seq)

```
get the sent packet info in sent_packets_map_
rtt=info.sent_ts−now
inflight_← inflight_−info.bytes
if rtt<min_rtt_ or min_rtt==0 then
    min_rtt_←rtt
    min_rtt_ts_←now
end if
if rtt<kSimilarMinRtt*min_rtt_ then
    min_rtt_ts_←now
end if
if seq>seq_at_backoff_ then
    if rtt<base_line_rtt_ then
        base_line_rtt_←rtt
        srtt_←rtt
    end if
    srtt_← (1 − α)*srtt_+α*rtt
end if
```

Algorithm 4 CheckIfCongestion

```
if srtt_==0 or base_line_rtt_== +∞ then
    return 0
end if
if mode_==ProbeBW and srtt_> β*base_line_rtt_ then
    return 1
end if
```

Algorithm 5 DrainExcessBuffer

Require:
(now, min_rtt_expired, congested)

```
if mode_! =ProbeRTT and (min_rtt_expired or congested) then
    mode_←ProbeRTT
    seq_at_backoff_←last_sent_packet_
    srtt_←0
    base_line_rtt_← +∞
    pacing_gain_← 0.75
    bdp_=bw_*min_rtt_
end if
if mode_==ProbeRTT then
    if inflight_< bdp_ then
        EnterProbeBwMode()
    end if
end if
```

Algorithm 6 Packet Scheduling Procedure

Require:
The id and length of newly generated packets

Ensure:
the map between packet id and path id

```
repeat
    Compute each path cost λ_p as equation (14)
    Get the path index j with the minimal path cost
    Schedule packet i to path j
    Increase the buffer pending length Q_j of path j
until all incoming packets are scheduled
```

ability makes it hard for interested readers to reproduce the algorithm. Thus code can be got at github.[4]

One more thing should be put forward. The padding packets carrying useless data should be sent for bandwidth probe purpose when there are no video packets available.

B. IMPLEMENTATION ON PACKET SCHEDULING ALGORITHM

The proposed packet scheduling algorithm is working on packet level but frame level. When an encoded video frame generates, the scheduler sends the packaged packets to local buffer of path with the minimal cost in (13). And the cost of path p in the proposed algorithm is defined as equation (14).

$$j = \arg\min_{p} \lambda_p \tag{13}$$

$$\lambda_p = \frac{RTT_p(t)}{2} + \frac{Q_p(t)}{c_p(t)} \tag{14}$$

$RTT_p(t)$ is round trip time. The first term $\frac{RTT_p(t)}{2}$ can be used as a reflection of queueing delay and propagation delay in network links. $Q_p(t)$ is occupied buffer length at sender. $c_p(t)$ is packet sending rate of the Pacer, and the rate is determined by congestion controller. When a packet is scheduled to path p, it will be buffered if the local buffer is not empty. The second term $\frac{Q_p(t)}{c_p(t)}$ is the expected waiting time before the buffered packet can be sent out. The detail on packet scheduling procedure is shown in Algorithm 6.

Even the scheduling algorithm cannot strictly guarantee in order of delivery of all packets at receiver due to some uncertain factors in the routing path, e.g., the occupied buffer length in routers, path reroute, it strives to achieve minimal arrival delay expectancy by choosing path with minimal cost first. Once these received packets can be reassembled as video frames, they would be delivered to upper layer as early as possible.

Since these packets belonging to a same frame may be sent to different paths, the receiver cannot figure out whether an incomplete frame is caused by packet lost or late arrival. At sub-path level, the feedback module would send acknowledgement packets to sender. Whether to retransmit the lost packets is totally depended on sender. The sent packets will be cached at sender at most 500 milliseconds. If a lost packet is found, it will be retransmitted immediately to the path with minimal transmission delay. The receiver decides the maximum time to wait for retransmitted packets. For key frame,

[4]https://github.com/SoonyangZhang/congestion-responce-experiment

receiver must wait for lost packets to be retransmitted and a complete frame can be rendered, or else the following frames that take the key frame as reference will not be decoded successfully. In none key frame case, the incomplete frame will be discarded if the waiting time exceeds a maximum threshold (500 milliseconds in experiment).

C. IMPLEMENTATION ON BENCHMARK ALGORITHMS

Three other packet scheduling algorithms Weighted Round Robin (WRR), EDCLD [28], and SFL [29] are implemented as benchmark algorithms for performance comparison purpose.

The WRR algorithm is applied for server load balance. The number of packets sent on path i are computed by equation (15). ABW_i is the available bandwidth of path i and n is the number of packets that wait to be scheduled.

$$path_load(i) = \lceil n\frac{ABW_i}{\sum_j ABW_j}\rceil \tag{15}$$

$$C_p(\psi_p) = D_p + (1-w)\frac{1}{\mu_p - \psi_p\lambda} + w\frac{q}{\mu_p} \tag{16}$$

Equation (16) is path cost defined in EDCLD. D_p is fixed propagation delay, μ_p is available bandwidth, q_p is occupied buffer length at sender side, ψ is packet scheduling ratio, λ is data generating rate, w is the weight. The cost function follows M/M/1 queue model when $w \leftarrow 0$. In experiment, w is set to 0.8. EDCLD will increase the number of packets sent on best path with lowest cost value and decrease scheduling ratio on worse path. To find optimal value of ψ, a quadratic equation $C_{p_{best}}(\psi_{best} + \Delta\psi) = C_{p_{worst}}(\psi_{worst} - \Delta\psi)$ needs to be solved. In an iteration, the packet scheduling ratio on these paths with cost between $C_{p_{best}}$ and $C_{p_{worst}}$ will remain unchanged.

The water filling algorithm in SFL takes available bandwidth and one way transmission delay into consideration. The basic idea is to make an equal aggregate delay by scheduling more packets to the path with smaller transmission delay and less packets to larger transmission delay path as shown in Figure 2, namely $D_i + d_i = D_j + d_j$. Here, d_i is one way delay of path i, D_i is the water depth. Through the adjustment of D_i, the total packets that all paths can be sent in a scheduling round should be equal to the incoming packets, $\sum_i D_i * ABW_i = \sum l$. The detail of the water filling packet algorithm is shown in Algorithm 7. For simplicity, when the current water of all path cannot hold all packets, the water depth of the path with the largest one way delay will increase by step of $\Delta D = \frac{L}{ABW}$ until all packets can be held, and L is the length of transmission unit.

V. EVALUATION

The performances of congestion control algorithm and packet scheduling algorithms are evaluated in this part. All experiments are running on ns-3.26.

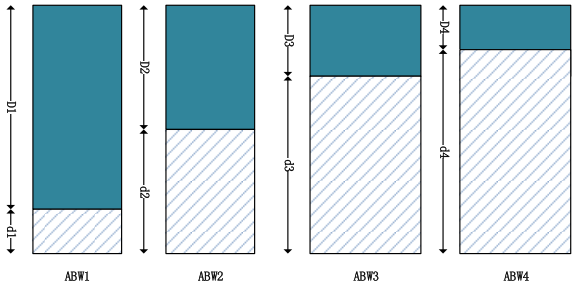


FIGURE 2. Watering filling method.

Algorithm 7 Water Filling Algorithm

Require:
The $path_info(ABW, OWD)$ every available path,
n, total available path,
The total size s and $packet_info(id, len)$ of newly generated packets

Ensure:
the map of packet id and path id

sort the $path_info$ according to owe way delay
$L \leftarrow packet_info[start_id].len$
$step \leftarrow \frac{L}{path_info[n-1].ABW}$
for $i \in [1, \infty)$ **do**
  $compensator \leftarrow i * step$
  $D \leftarrow path_info[n-1].OWD + compensator$
  $water \leftarrow 0$
  for $j \in [0, n)$ **do**
    $owd \leftarrow path_info[j].OWD$
    $abw \leftarrow path_info[j].ABW$
    $path_info[j].bytes = \frac{(D-owd)*abw}{1000*8}$
    $water \leftarrow water + path_info[j].bytes$
  end for
  if $water \geq s$ **then**
    break
  end if
end for
$i \leftarrow 0$
for $k \in [start_id, end_id]$ **do**
  while $i \neq n-1$ **and** $path_info[i].bytes < 0$ **do**
    $i \leftarrow i + 1$
  end while
  Schedule packet k to path i
  $len = packet_info[start_id].len$
  $path_info[i].bytes \leftarrow path_info[i].bytes - len$
end for

A. PERFORMANCE OF THE CONGESTION CONTROL ALGORITHM

In order to test the performance of Delay-BBR algorithm, a point to point topology is built with link configuration shown in Table 1. The parameters are link capacity (in unit of Mbps), one way propagation delay (in unit of millisecond),

TABLE 1. Network configuration.

Case	Bandwidth	One Way Delay	Queue buffer
1	3Mbps	100ms	3Mbps*300ms
2	3Mbps	100ms	3Mbps*400ms
3	3Mbps	100ms	3Mbps*600ms
4	4Mbps	100ms	4Mbps*300ms
5	4Mbps	100ms	4Mbps*400ms
6	4Mbps	100ms	4Mbps*600ms
7	5Mbps	100ms	5Mbps*300ms
8	5Mbps	100ms	5Mbps*400ms
9	5Mbps	100ms	5Mbps*600ms

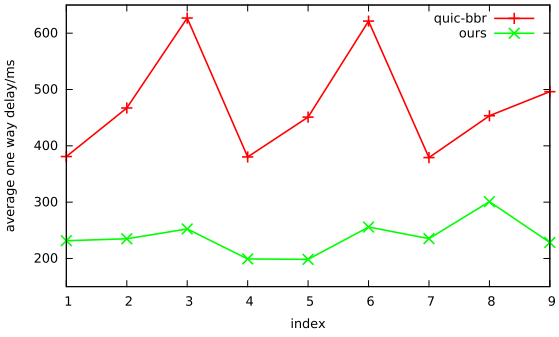


FIGURE 3. Average packet one way transmission delay.

and buffer length in routers. We get the BBR algorithm implemented in google QUIC running on ns3 platform for comparison purpose. The experiments are running about 300 seconds.

On each experiment, three flows exploiting same congestion control algorithm are initiated at different time points. The second flow is started 40s late after the first flow and the third flow is started at 80s. At sender side, the packet sending rate is traced. One way transmission delay and the sequence number of loss packets are recorded at receiver. The average one way transmission delay from all received packets in each test case is plotted in Figure 3. The packets loss rate of all 9 experiments is showed in Figure 4.

One way delay signal is composed by propagation delay and queueing delay, which can be used to indicate the dynamic of occupied buffer in routers. Delay-BBR algorithm achieves lower packet transmission delay and extremely lower packet loss rate compared with QUIC-BBR. As routers are configured with more buffer (Case 1, 2, 3), the packet loss rate decreases and transmission delay increases in QUIC-BBR. The QUIC-BBR flows would occupy much buffer resource in routers.

The rate dynamics of QUIC-BBR flows and Delay BBR flows are plotted in Figure 5 and Figure 6 under the link configuration in Case 2. The rates of QUIC-BBR flows can maintain fairness on average but show a wide range of variation from 500kbps to 1500bps due to competition. The rates of Delay-BBR flows show stability feature and can also converge to the fairness line near 1Mbps in Figure 6. A single test may be not persuasive, the results in Figure 8 further show such rate fairness property in different experiments. This rate stability property makes the proposed algorithm fit for video transmission.

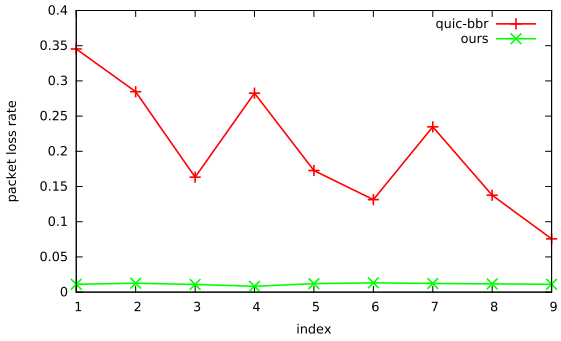


FIGURE 4. Average packet loss rate.

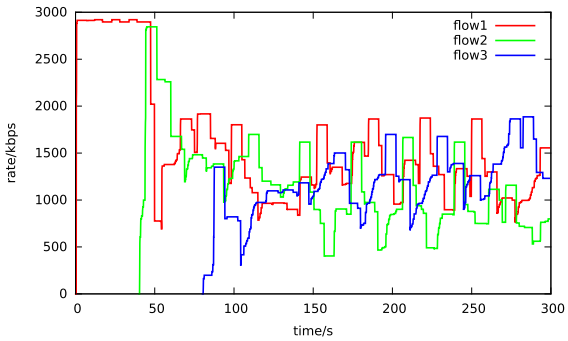


FIGURE 5. Rate dynamics of QUIC-BBR flows.

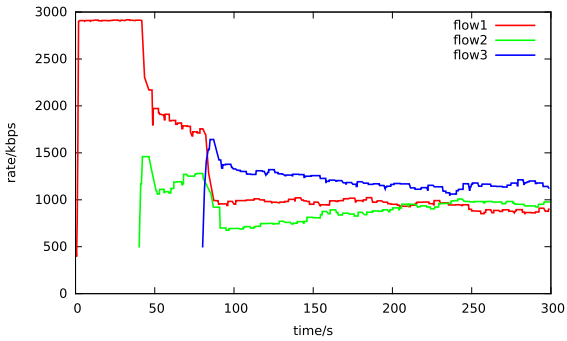


FIGURE 6. Rate dynamics of delay-BBR flows.

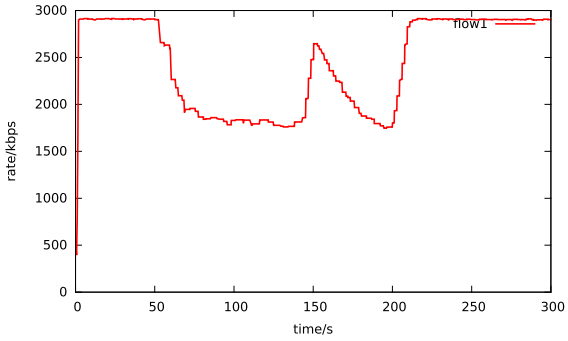
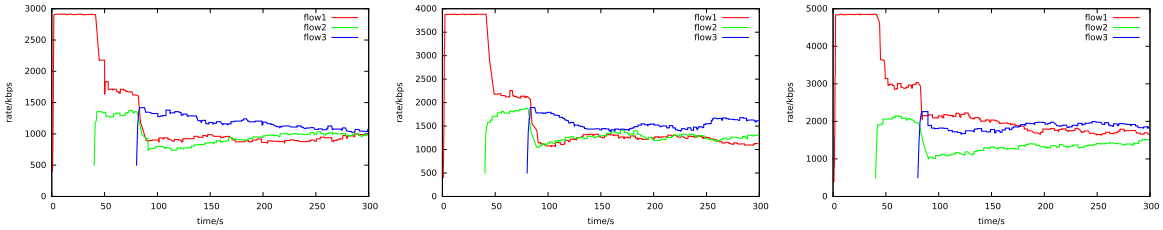


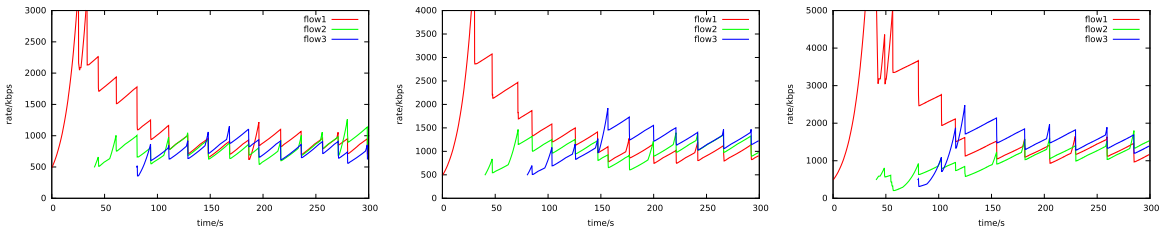
FIGURE 7. Rate dynamic of Delay-BBR flow.

The reason for high packet loss rate and longer transmission delay of QUIC-BBR is that all flows tend to insist the maximum bandwidth sampled within 10 RTTs and the



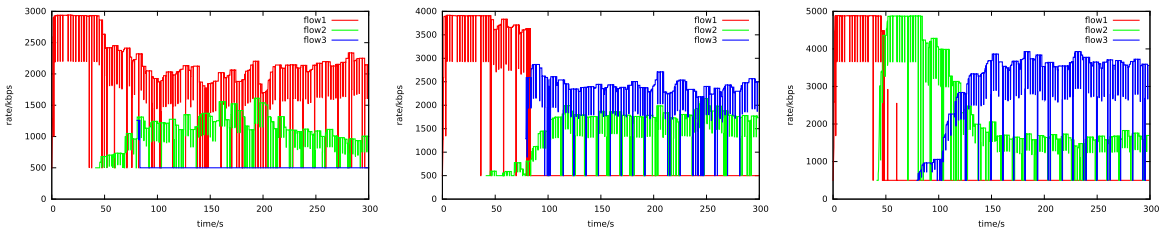
(a) (b) (c)

FIGURE 8. Rate dynamics of Delay-BBR flows. (a) C1. (b) C4. (c) C7.



(a) (b) (c)

FIGURE 9. Rate dynamics of WebRTC-GCC flows. (a) C1. (b) C4. (c) C7.



(a) (b) (c)

FIGURE 10. Rate dynamics of WebRTC-BBR flows. (a) C1. (b) C4. (c) C7.

total sending rate would exceed the link capacity as shown in Figure 5. In such situation, the occupied link buffer cannot be drained in time and extra sent packet would be buffered in routers. When link buffer overflows, packet loss will happen. The modified algorithm responses delay signal as detailed in algorithm implementation part. A lower queueing delay and lower packet loss rate is thus achieved.

Traditionally, the delay response congestion control algorithms such as Vegas [35] tend to get starvation when sharing links with loss based congestion control algorithms. To verify the proposed algorithm does not suffer such problem, an experiment is conducted that a TCP Reno flow and a Delay-BBR flow share routing in Case 2 configuration. The TCP Reno flow is started at 50s and ended at 200s. The rate dynamic of the Delay-BBR flow is shown in Figure 7. The flow can maintain reasonable throughput and indeed yields bandwidth during the presence of TCP Reno flow. After rate back-off of TCP Reno flow and occupied buffer is emptied, Delay-BBR flow has rate increase tendency near 150s. When TCP Reno flow stops, Delay-BBR flow increases rate rapidly to the point close to link capacity. Thanks to the reset operation of srtt signal when the state is changed to ProbeRTT and srtt would be resampled in state ProbeBW, which could prevent the control state being changed to ProbeRTT too frequently to yield too much bandwidth.

The default congestion control algorithm in WebRTC is GCC. BBR algorithm is also implemented in WebRTC, but there is no report available on its performance in real network until now. In our previous work [36], performance comparison on GCC, NADA, SCReAM and WebRTC-BBR is conducted on ns3. Here, the performance of GCC, WebRTC-BBR and our optimized version is tested with link configuration in Case 1, 4, 7. In each case, three flows are injected into network and the experiments are running about 300s. And the dynamics on sending rates of all three flows with different congestion control algorithms are shown in Figure 8, 9 and 10. Both GCC flows and Delay-BBR flows can converge to fairness bandwidth line. While the rates of the WebRTC-BBR flows show oscillation and a flow only maintains a minimal rate due to two other flows occupy much bandwidth resource as Figure 10 shows.

Further, the average one way transmission delay and packet loss rate are given in Table 2. Some conclusions can be made here. GCC flows can maintain the lowest transmission delay

TABLE 2. Transmission delay and loss rate of releated algorithms.

Case	GCC	WebRTC-BBR	Delay-BBR
	(OWD(ms), Loss(%))		
1	(118.20, 9.97)	(369.82, 11.59)	(231.51, 1.11)
4	(117.86, 10.15)	(367.90, 10.06)	(199.11, 0.83)
7	(124.93, 11.57)	(355.02, 10.26)	(235.19, 1.23)

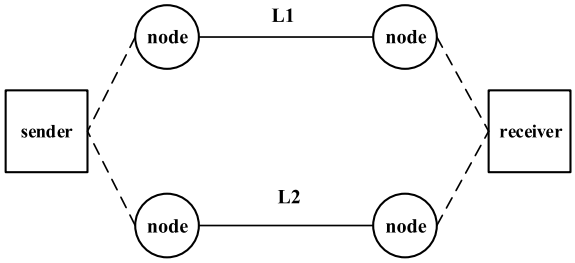


FIGURE 11. Multipath topology.

TABLE 3. Link configuration.

Case	L1	L2
	(BW, OWD, Q)	
1	(4, 100, 200)	(4, 100, 200)
2	(3, 100, 200)	(2, 150, 200)
3	(3, 100, 200)	(2, 100, 200)
4	(4, 100, 200)	(2, 50, 200)
5	(4, 50, 200)	(2, 100, 200)
6	(4, 50, 200)	(4, 50, 200)
7	(3, 100, 200)	(3, 100, 200)
8	(4, 100, 200)	(3, 150, 200)
9	(4, 150, 200)	(3, 50, 200)
10	(2, 100, 200)	(3, 100, 200)

but have considerable packet loss rate about 10%. Delay-BBR flows achieve lowest packet loss rate, which can almost be negligible but have higher average packet transmission delay when compared with GCC.

B. PERFORMANCE OF THE PACKET SCHEDULE ALGORITHM

The performances of these packet scheduling algorithms are analyzed. All these packets scheduling algorithms work under the constraint of the proposed congestion control algorithm. A simple multipath topology is built as Figure 11. Only two paths are tested. The link configuration is shown in Table 3. The parameters on each link are capacity (BW, in unit of Mbps), one way transmission delay (OWD, in unit of milliseconds), link queue length (Q, in unit of milliseconds). When buffered packets exceed the maximum queue length (Q*BW), subsequent packets will be discarded due to the deployment of droptail queue management. There are totally 10 experiments and all experiments are running about 300 seconds.

In real network, a routing path can be shared by many flows. Besides the multipath session, two flows on L1 and one flow on L2 with the same congestion control algorithm are initiated simulate bandwidth competition among flows. An ideal video encoder is applied, in which the video frame bitrate can be changed immediately as the requirement of congestion controller. A maximum video bitrate 2Mbps is

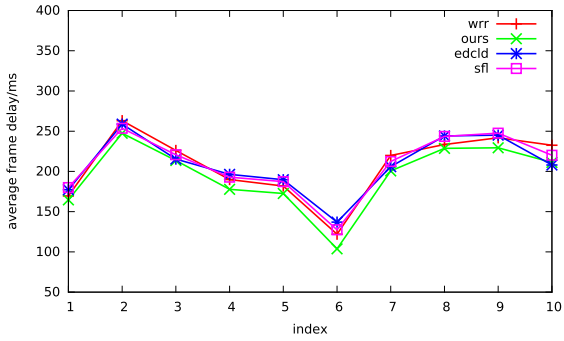


FIGURE 12. The average frame transmission delay.

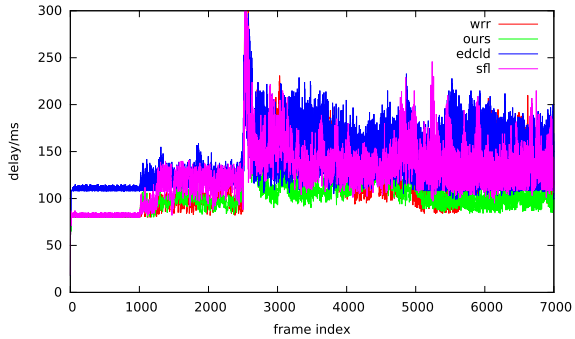


FIGURE 13. The frame delay with link configure 6.

set and the video fame generating bitrate will not increase further if total available bandwidth is above the maximum bitrate. During simulation, the frame transmission delay (the interval between the time point that a frame can be decoded at receiver and the time point that an encoded frame is generated at sender) is traced.

The metrics like PSNR, SSIM to evaluate image quality seem only related to the allocated bitrate of encoded frames. These metrics are related to the throughput of congestion control algorithm, since the throughput of congestion control algorithm is used as reference to adjust the output bitrate of the video encoder. All the packet scheduling algorithm are tested under the proposed congestion control algorithm. There may not obvious difference if this two metrics are applied for evaluation. Hence the average frame transmission delay is used for such purpose. There are about 7000 frames generated by encoder during simulation. The average frame transmission delay of different packet scheduling algorithm is showed in Figure 12. In most cases, the proposed algorithm achieves lower average frame delay compared to these reference algorithms. Especially, each frame transmission delay in experiment case 6 is shown in Figure 13. The delay increase tendency in Figure 13 is the result of network link congestion due to the link buffer occupation when flows competing for bandwidth.

VI. CONCLUSION

In this work, a multipath transmission framework for real time video traffic is proposed. Under the multipath

transmission context, there are two basic research points: congestion control algorithm and packet scheduling algorithm. The congestion control algorithm is to make sender maintain a throughput as close as the available bandwidth while avoiding leading network into congestion. Since traditional AIMD congestion control algorithm is not suitable for real time video transmission due to its sharp rate reduction when a single packet is lost and high transmission delay caused by link buffer occupation. Implementing congestion control algorithms on application layer for real time video traffic becomes a trend. The most difficult part to design a new congestion control algorithm is the fairness property, namely to guarantee these flows traversing a common bottleneck can gain similar throughput with limited feedback information. And the packet scheduling algorithm is to take advantages of the paths heterogeneity to provide better service.

Extensive experiments are conducted to compare the performances of GCC, QUIC-BBR, WebRTC-BBR and Delay-BBR. BBR implemented in QUIC for bulk data transfer will cause high packets loss rate and large transmission delay when flows competing for bandwidth. There is considerable packet loss rate when GCC flows competing for bandwidth. Delay-BBR algorithm has nice properties such as bandwidth fairness, rate stability, lower packet loss rate and lower transmission delay. To make the BBR congestion control algorithm achieve the claimed optimal control point, in which the sender can achieve the maximum throughput while minimizing end to end latency at the same time, is still an open problem. Even Delay-BBR can achieve lower one way transmission delay compared with QUIC-BBR and WebRTC-BBR, it is considerable higher when compared with GCC. To further reduce delay, which means to minimize queue occupation in routers is one direction of future work. Of course, the fair bandwidth property should be maintained at the same time.

A multipath packet scheduling algorithm induced from utilization maximization theory is proposed. In essence, it can be regarded as earliest first algorithm but takes consideration of local occupied buffer length, available bandwidth and transmission delay on each path. When working in collaboration with the proposed congestion control algorithm, it achieves better performance in term of average frame delivery delay compared with benchmark algorithms.

In real network, different flows traversing the same bottleneck may exploit different congestion control algorithms and some flows over UDP even do not implement any rate control mechanism at all. Thus link congestion is unavoidable and packets loss events happen frequently. For bulk data transfer, the packet loss can be compensated by retransmission. But for real time video traffic, retransmission may not be effective if the packet arriving time has exceeded the video frame rendering deadline. In future, the combination of multipath transmission with forward error correction (FEC) to improve QoE for real time video will be considered.

REFERENCES

[1] "Cisco visual networking index: Forecast and trends, 2017–2022," Cisco, San Jose, CA, USA, White Paper, Feb. 2019. [Online]. Available: https://www.cisco.com/c/en/us/solutions/collateral/service-provider/visual-networking-index-vni/white-paper-c11-741490.html

[2] V. Singh, S. Ahsan, and J. Ott, "MPRTP: Multipath considerations for real-time media," in *Proc. 4th ACM Multimedia Syst. Conf.*, Mar. 2013, pp. 190–201.

[3] C. Raiciu, M. Handley, and D. Wischik, *Coupled Congestion Control for Multipath Transport Protocols*, document, RFC 6356, Internet Requests for Comments, Oct. 2011. [Online]. Available: https://www.rfc-editor.org/info/rfc6356

[4] R. Khalili, N. Gast, M. Popovic, and J.-Y. Le Boudec, "MPTCP is not pareto-optimal: Performance issues and a possible solution," *IEEE/ACM Trans. Netw.*, vol. 21, no. 5, pp. 1651–1665, Oct. 2013. doi: 10.1109/TNET.2013.2274462.

[5] Y. Cao, M. Xu, and X. Fu, "Delay-based congestion control for multipath TCP," in *Proc. 20th IEEE Int. Conf. Netw. Protocols (ICNP)*, Oct. 2012, pp. 1–10.

[6] A. Frommgen, T. Erbshäußer, A. Buchmann, T. Zimmermann, and K. Wehrle, "ReMP TCP: Low latency multipath TCP," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2016, pp. 1–7.

[7] Y. E. Guo, A. Nikravesh, Z. M. Mao, F. Qian, and S. Sen, "Accelerating multipath transport through balanced subflow completion," in *Proc. 23rd Annu. Int. Conf. Mobile Comput. Netw.*, Oct.2017, pp. 141–153.

[8] Y.-S. Lim, E. M. Nahum, D. Towsley, and R. J. Gibbens, "ECF: An MPTCP path scheduler to manage heterogeneous paths," in *Proc. 13th Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2017, pp. 147–159. doi: 10.1145/3143361.3143376.

[9] H. Shi *et al.*, "STMS: Improving MPTCP throughput under heterogeneous networks," in *Proc. USENIX Annu. Tech. Conf.*, Jul. 2018, pp. 719–730.

[10] T. Shreedhar, N. Mohan, S. K. Kaul, and J. Kangasharju. (2018). "QAware: A cross-layer approach to MPTCP scheduling." [Online]. Available: https://arxiv.org/abs/1808.04390

[11] S. Ferlin, O. Alay, O. Mehani, and R. Boreli, "BLEST: Blocking estimation-based MPTCP scheduler for heterogeneous networks," in *Proc. IFIP Netw. Conf. (IFIP Netw.) Workshops*, May 2016, pp. 431–439.

[12] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, Aug. 1988.

[13] G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo, "Analysis and design of the google congestion control for Web real-time communication (WebRTC)," in *Proc. 7th Int. Conf. Multimedia Syst.*, May 2016, p. 13.

[14] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-based congestion control," *Queue*, vol. 14, no. 5, p. 50, Sep./Oct. 2016.

[15] E. Rescorla. (Dec. 2014). Congestion control requirements for RMCAT. Internet Engineering Task Force. Internet-Draft. [Online]. Available: https://tools.ietf.org/html/draft-ietf-rmcat-cc-requirements-09

[16] L. Xu, K. Harfoush, and I. Rhee, "Binary increase congestion control (BIC) for fast long-distance networks," in *Proc. INFOCOM*, Mar. 2004, pp. 2514–2524.

[17] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, Jul. 2008.

[18] V. Cerf, V. Jacobson, N. Weaver, and N. Weaver, "BufferBloat: What's wrong with the Internet?" *Queue*, vol. 9, no. 12, p. 10, 2011.

[19] X. Zhu *et al.* (2018). NADA: A unified congestion control scheme for real-time media. Internet Engineering Task Force. Internet-Draft. [Online]. Available: https://tools.ietf.org/html/draft-ietf-rmcat-nada-07

[20] I. Johansson and Z. Sarker. *Self-Clocked Rate Adaptation for Multimedia*, document, RFC 8298, Internet Requests for Comments, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8298.txt

[21] N. Kuhn, E. Lochin, A. Mifdaoui, G. Sarwar, O. Mehani, and R. Boreli, "DAPS: Intelligent delay-aware packet scheduling for multipath transport," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2014, pp. 1222–1227.

[22] C. Cetinkaya and E. W. Knightly, "Opportunistic traffic scheduling over multiple network paths," in *Proc. IEEE INFOCOM*, Mar. 2004, pp. 1928–1937.

[23] V. J. Ribeiro, R. H. Riedi, R. G. Baraniuk, J. Navratil, and L. Cottrell, "pathChirp: Efficient available bandwidth estimation for network paths," in *Proc. Passive Active Meas. Workshop*, Apr. 2003, p. 1.

[24] I. Norros, "A storage model with self-similar input," *Queueing Syst.*, vol. 16, nos. 3–4, pp. 387–396, Sep. 1994.

[25] X. Zhu, P. Agrawal, J. Pal Singh, T. Alpcan, and B. Girod, "Rate allocation for multi-user video streaming over heterogenous access networks," in *Proc. 15th ACM Int. Conf. Multimedia*, Sep. 2007, pp. 37–46.
[26] J. Wu, C. Yuen, M. Wang, and J. Chen, "Content-aware concurrent multipath transfer for high-definition video streaming over heterogeneous wireless networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 3, pp. 710–723, Mar. 2016.
[27] V. Bui, W. Zhu, A. Botta, and A. Pescape, "A Markovian approach to multipath data transfer in overlay networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 21, no. 10, pp. 1398–1411, Oct. 2010.
[28] S. Prabhavat, H. Nishiyama, N. Ansari, and N. Kato, "Effective delay-controlled load distribution over multipath networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 10, pp. 1730–1741, Oct. 2011.
[29] J. Wu, X. Qiao, Y. Xia, C. Yuen, and J. Chen, "A low-latency scheduling approach for high-definition video streaming in a heterogeneous wireless network with multihomed clients," *Multimedia Syst.*, vol. 21, no. 4, pp. 411–425, Jul. 2015.
[30] A. Aggarwal, S. Savage, and T. Anderson, "Understanding the performance of TCP pacing," in *Proc. IEEE INFOCOM*, Mar. 2000, pp. 1157–1165.
[31] D. Wei, P. Cao, S. Low, and C. Eas. (2006). *TCP Pacing Revisited*. [Online]. Available: http://people.cs.pitt.edu/~ihsan/pacing_cal.pdf
[32] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate control for communication networks: Shadow prices, proportional fairness and stability," *J. Oper. Res. Soc.*, vol. 49, no. 3, pp. 237–252, Mar. 1998.
[33] S. H. Low and D. E. Lapsley, "Optimization flow control. I. Basic algorithm and convergence," *IEEE/ACM Trans. Netw.*, vol. 7, no. 6, pp. 861–874, Dec. 1999.
[34] F. Paganini and E. Mallada, "A unified approach to congestion control and node-based multipath routing," *IEEE/ACM Trans. Netw.*, vol. 17, no. 5, pp. 1413–1426, Oct. 2009.
[35] L. S. Brakmo and L. L. Peterson, "TCP Vegas: End to end congestion avoidance on a global Internet," *IEEE J. Sel. Areas Commun.*, vol. 13, no. 8, pp. 1465–1480, Oct. 1995.
[36] S. Zhang. (2018). "Congestion control for RTP media: A comparison on simulated environment." [Online]. Available: https://arxiv.org/abs/1809.00304



SONGYANG ZHANG is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, Northeastern University, China. His current research interests include multipath transmission for real time traffic, network congestion control, and overlay networks.



WEIMIN LEI received the B.E. degree in computer software from Nankai University, in 1995, the M.E. degree in computer software from the Chinese Academy of Sciences, in 1992, and the Ph.D. degree from the Dalian University of Technology, in 1999. He is currently a Professor with the School of Computer Science and Engineering, Institute of Communication and Information System, Northeastern University, China. He has published over 80 papers. His recent research interests include real-time multipath transmission optimization, real-time signal processing, and future network architecture.



WEI ZHANG is currently a Lecturer with the School of Computer Science and Engineering, Institute of Communication and Information System, Northeastern University, China. Her current research interests include multimedia communication, protocols and services in next generation networks, and network architecture of cloud computing.



YUNCHONG GUAN is currently pursuing the Ph.D. degree in computer application technology with the School of Computer Science and Engineering, Northeastern University, China. His research interests include network architecture and the multipath routing of multipath transmission framework, SDN, overlay networks, and the signaling control of IP multimedia communication.



HAO LI received the B.Eng. degree in communication engineering from Northeastern University, Shenyang, China, in 2014, where he is currently pursuing the Ph.D. degree. His current research interests include transmission control, network architecture, video coding, and image processing.